

AWS - Fundamentos, EC2, S3, VPC y RDS

AWS Fundamentals, Presupuestos y Seguridad (IAM)

¿Qué es el concepto?

La nube es rentar infraestructura (servidores, redes) por hora a través de una API. Según el modelo que ecogemos de responsabilidad, AWS se encarga de que hardware, energía, red física, funcionen, pero yo soy responsable de lo que hago adentro: escoger mi sistema operativo, los puertos que abro y mis credenciales. Además, antes de lanzar cualquier recurso, tenemos que configurar presupuestos y gestionar accesos seguros (IAM) para no usar el usuario root principal.

El proceso que realicé:

Lo primero que hice fue crear un budget por un límite de \$5 USD mensuales, configurando alertas que me avisen por correo si llego al 80% o al 100%.

Después, tenía que estar segura de no trabajar con la cuenta root para mis prácticas. Entré a IAM y creé un usuario llamado taskflow-admin con permisos de administrador, y guardé su URL de inicio de sesión y credenciales. Después, cerré sesión y todo el trabajo lo hice desde esta cuenta IAM en la región de Virginia (us-east-1).

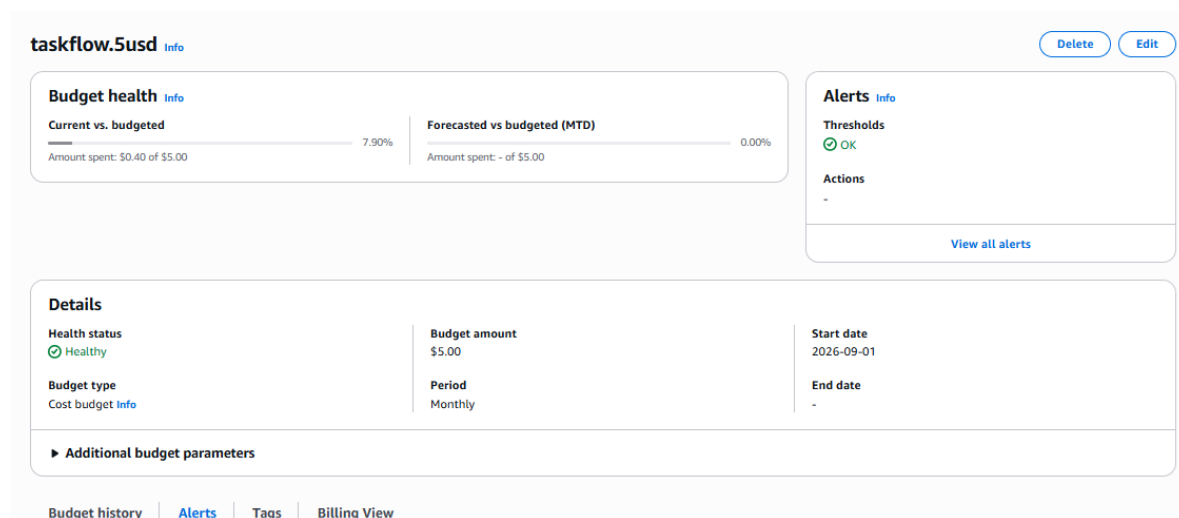


Ilustración 1- La consola de AWS mostrando la página de Budgets con el presupuesto de "taskflow-5usd" activo

Alerts (3) Info

View detailsDelete alertEdit alerts

Review any exceeded thresholds and any pending actions below from your alerts.

Actual cost > 85%

Definition

When your actual cost is greater than 85% (\$4.25) of your budgeted amount (\$5.00), the alert threshold will be exceeded.

Threshold

Not exceeded

Actions

-

Forecasted cost > 100%

Definition

When your forecasted cost is greater than 100% (\$5.00) of your budgeted amount (\$5.00), the alert threshold will be exceeded.

Threshold

Not exceeded

Actions

-

Actual cost > 100%

Definition

When your actual cost is greater than 100% (\$5.00) of your budgeted amount (\$5.00), the alert threshold will be exceeded.

Threshold

Not exceeded

Actions

-

Ilustración 2 -: La consola de AWS mostrando la página de Budgets y sus dos alertas configuradas

Comentarios personales:

La práctica de que separemos el usuario root del usuario IAM me pareció una muy buena práctica de seguridad. También sentí que tenía que tener cuidado al hacer el Budget y darme cuenta que una instancia o recursos que se olvidan nos pueden salir caros, así que armar el presupuesto es una gran practica de prevención.

Ilustración 3 - el arte ASCII de Amazon Linux y la salida del comando free -m

Comentarios personales:

Entendí que nunca debemos dejar el puerto 22 abierto a "Anywhere (0.0.0.0/0)" porque los bots escanean puertos abiertos muy rápido. También fue interesante ver que si no cambias bien los permisos del archivo .pem (con el chmod), el SSH nos rebota la conexión. Levantar un servidor en solo unos minutos fue de las cosas que más me gustaron de la práctica.

Respondiendo a la pregunta del proyecto: ¿Qué habría pasado si dejabas el SSH abierto al mundo?

Si hubiera dejado el puerto 22 abierto a 0.0.0.0/0 (Anywhere) en el Security Group, los bots automatizados habrían encontrado mi servidor en minutos o horas. Van a intentar forzar la entrada probando contraseñas, lo que compromete por completo la seguridad de mi instancia. Por eso tenemos que limitarlo a "My IP".

Amazon S3 (Almacenamiento y Transferencia de Artefactos)

¿Qué es el concepto?

S3 es un servicio de almacenamiento de objetos. Funciona por medio de "Buckets" (contenedores con nombres únicos). Aquí pude alojar los artefactos de nuestra aplicación (el .jar compilado) y mantenerlos privados.

El proceso que realicé:

Creé un bucket privado llamado taskflow-artefactos-inzunza bloqueando el acceso público. Compilé mi proyecto en mi laptop con `mvn -q -DskipTests package` para haer el archivo `taskflow-api-3.0.0.jar`. Subí este archivo al bucket de S3.

Para la seguridad, intenté abrir la "Object URL" del archivo y como esperaba, me arrojó un error de `AccessDenied`. Luego generé una "Presigned URL" válida por 5 minutos, y con esa sí logré descargarlo.

Finalmente, en vez de descargarlo desde S3, decidí pasarlo directo de mi laptop a mi servidor EC2 usando el comando seguro de copiado por red: `scp -i taskflow-key.pem target/taskflow-api-*.jar ec2-user@ 54.83.122.102:~/taskflow-api.jar`. Verifiqué que el archivo llegó bien corriendo un `sha256sum`.

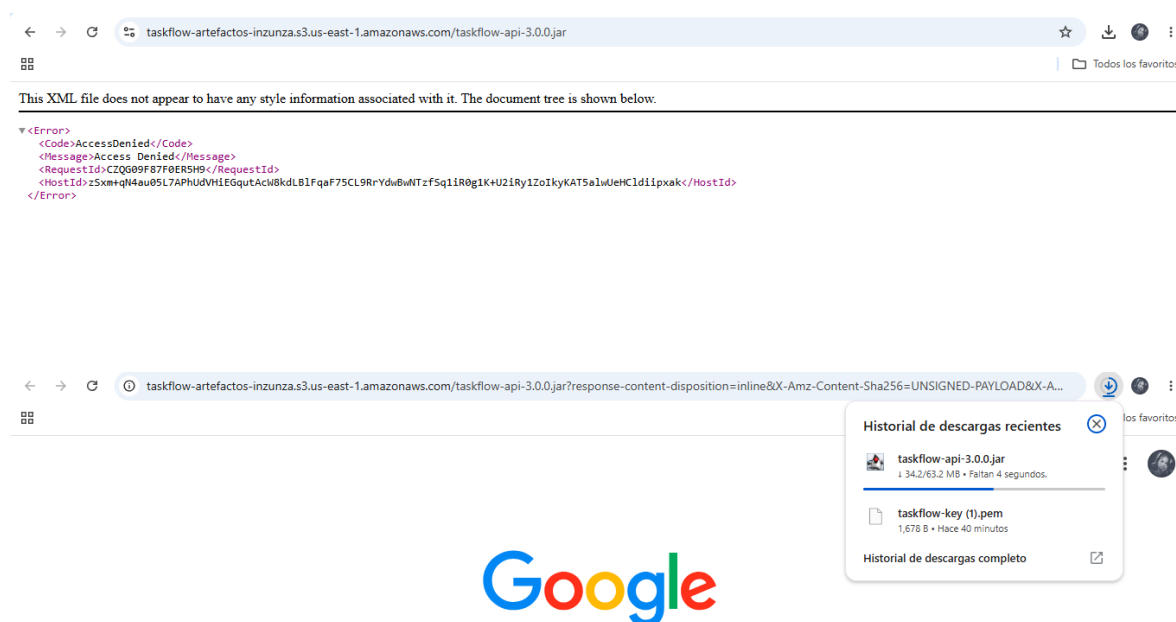


Ilustración 4 - error XML de AccessDenied en la Object URL y la descarga usando la Presigned URL

Comentarios personales:

Me llamó mucho la atención por qué compilamos el .jar localmente y no directo en el servidor y ya después comprendí que como la EC2 t3.micro solo tiene 1 GB de RAM, si ponemos a Maven a empaquetar ahí, se queda sin memoria y el proceso se va a terminar.

Amazon RDS y Conexión de la API (Bases de Datos en la Nube)

¿Qué es el concepto?

RDS gestiona bases de datos sin que nos preocupemos del sistema operativo o actualizaciones. Para que nuestra EC2 se comunique con RDS, deben existir reglas en el Security Group de la base de datos para el paso del tráfico desde el servidor.

El proceso que realicé:

Creé una base de datos PostgreSQL (db.t4g.micro con 20 GB de almacenamiento SSD gp3) llamada taskflow-db. Le deshabilité el acceso público.

Configuré el Security Group del RDS (taskflow-rds-sg). Fui a las Inbound Rules y agregué una regla en el puerto 5432, pero en el "Source" (origen) no puse una IP fija, sino que busqué y seleccioné el Security Group de mi EC2 (taskflow-ec2-sg).

Después, en la terminal de mi EC2, generé un secreto JWT seguro con openssl rand -hex 32 y arranqué la API inyectando las variables de entorno.

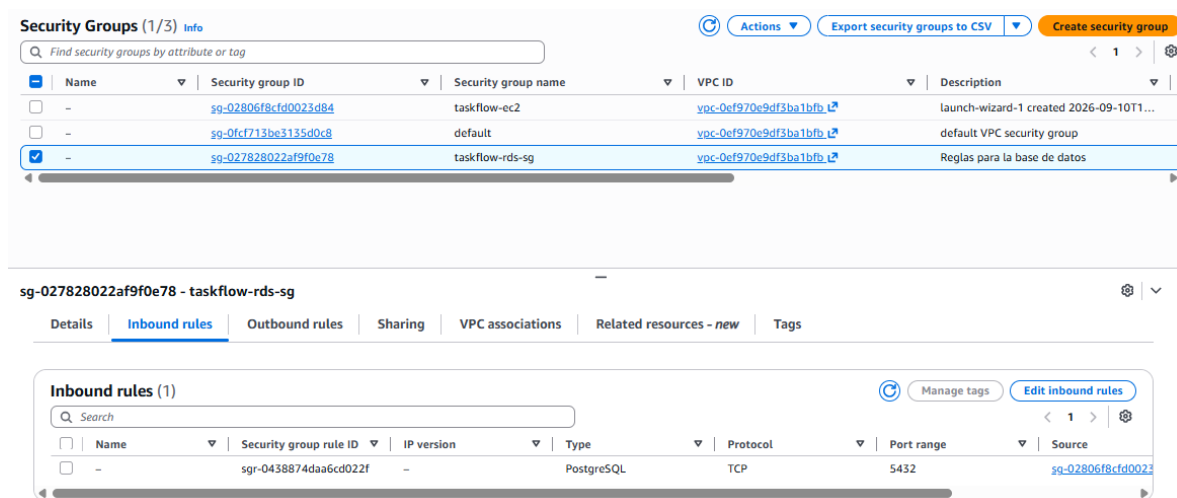


Ilustración 5- La configuración de Inbound Rules del Security Group de RDS

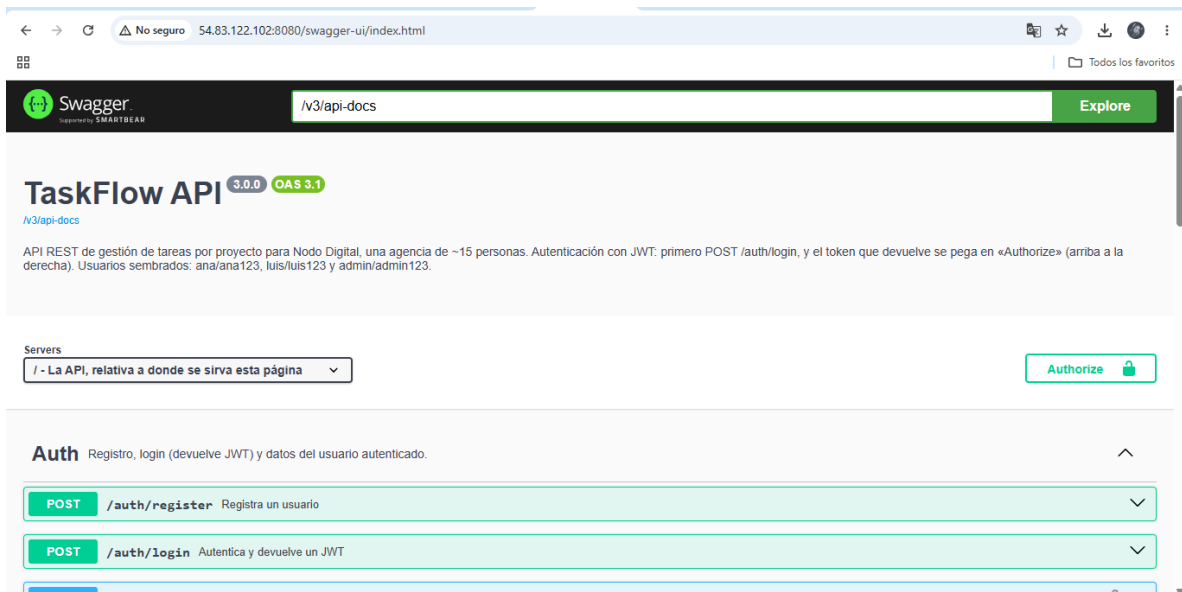


Ilustración 6 - Navegador abriendo el Swagger UI desde tu IP pública en el puerto 8080

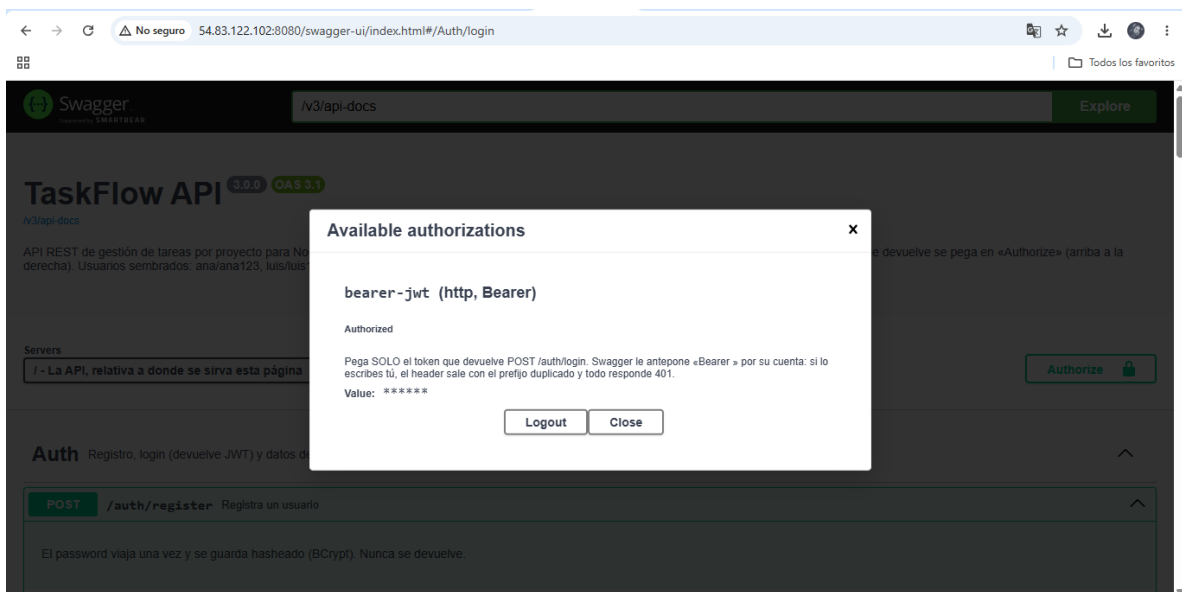


Ilustración 7- loguenadome tras conectar con RDS

Comentarios personales:

Aquí fue donde utilicé el diagnóstico de red. Al principio, si intentaba conectar la API y miraba el log (grep -n "Caused by" app.log), notaba el error Connect timed out. Entendí que eso significa que un firewall que me faltaba arreglar el Security Group del RDS.

La gran lección del día fue ver que pasé de usar una base local de prueba a una base PostgreSQL de producción en un servidor real sin modificar mi código en Java y usar el mismo archivo .jar original, únicamente inyectándole nuevas variables de entorno.

Respondiendo a la pregunta del proyecto: ¿Por qué el RDS no tiene IP pública, y cómo llega entonces la EC2 a él?

El RDS no tiene IP pública por seguridad, las bases de datos no deben estar expuestas a internet. La EC2 puede llegar a la base de datos porque las dos viven en la misma red de aws. La conexión se da gracias a que configuré el Security Group del RDS para permitir el tráfico en el puerto 5432 del Security Group de mi EC2, con esto se crea el túnel entre ambos recursos.